

Pharmacy System

Requirements Summary

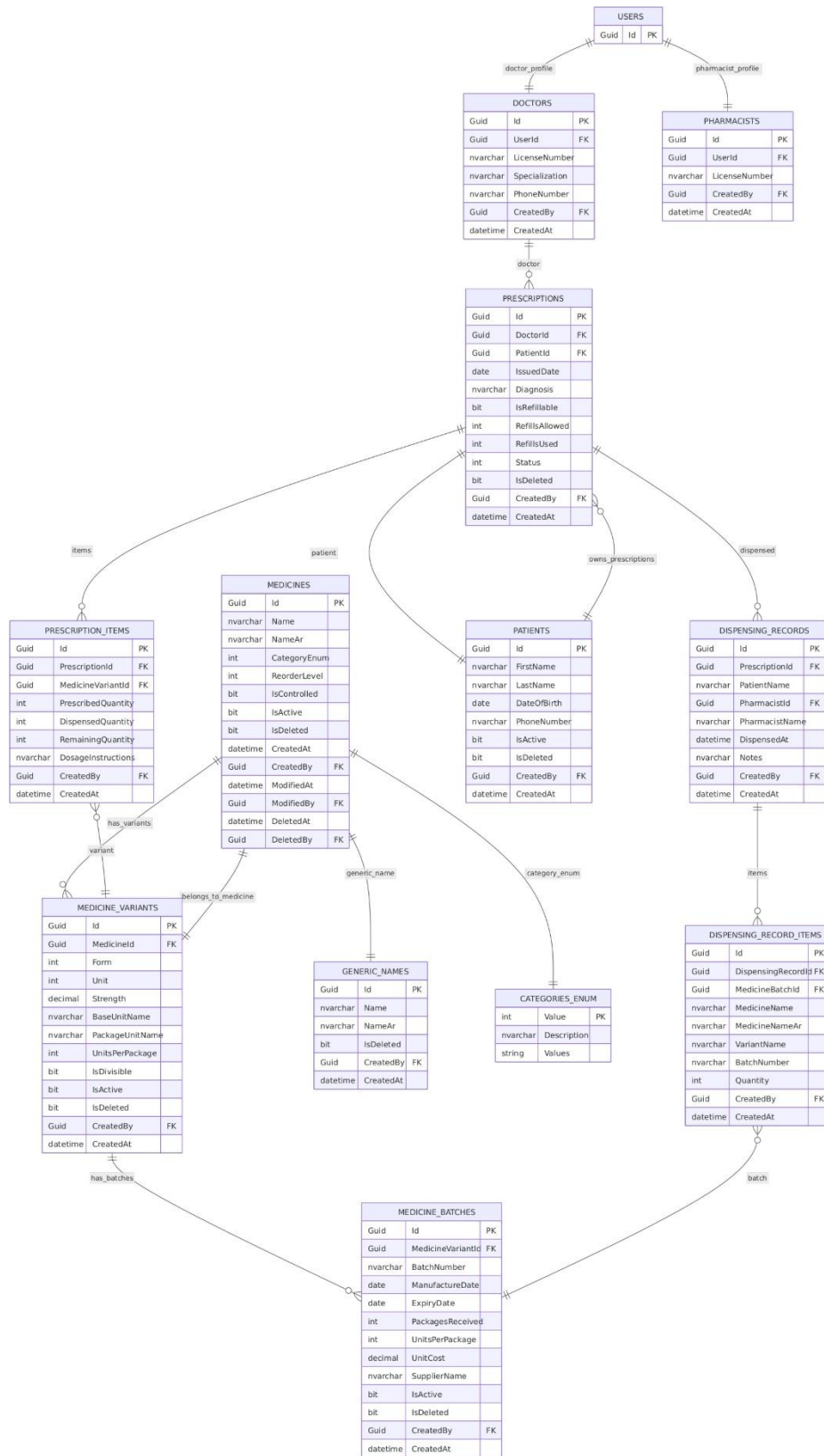
1. Functional Requirements

- **Medicine Batch Management** – Record batch number, manufacture date, expiry date, quantity received, available quantity, and unit cost.
- **Expiry Alerts** – Provide expiry statuses (**Critical, Warning, Safe, Expired**) based on predefined expiry thresholds.
- **Low-Stock Warnings** – Identify medicines whose available quantity is below their reorder level.
- **Medicine Dispensing** – Allow pharmacists to dispense prescribed medicines and update stock quantities accordingly.
- **Atomic Dispensing & Concurrency** – Ensure dispensing maintains stock consistency and prevents conflicting concurrent updates.
- **Prevent Expired Medicine Dispensing** – Reject dispensing from expired batches with HTTP 422.
- **Stock Sufficiency Check** – Prevent dispensing when available stock is insufficient for the prescribed quantity.
- **Refill Rules** – Allow refills only when the prescription is refillable, its status is **Fully Dispensed**, and used refills are below the allowed limit.
- **Ownership-Based Access Control** – Restrict doctors to their own prescriptions and enforce role-based permissions.
- **Audit Log** – Record relevant changes including the actor, timestamp, and old/new values, with admin-only access.
- **Soft Delete** – Support logical deletion of applicable records.
- **Data Export** – Allow supported data to be exported to Excel and PDF.
- **Real-Time Notifications** – Send notifications for prescription creation, dispensing, low-stock conditions, and near-expiry batches.
- **File Upload** – Support JPEG, PNG, and PDF uploads with a maximum file size of 5 MB.
- **Patient Identity** – Use a normalized Saudi mobile phone number as the patient's unique identifier.
- **Localization** – Support English and Arabic, including RTL layout for Arabic and language selection through `Accept-Language`.

2. Non-Functional Requirements

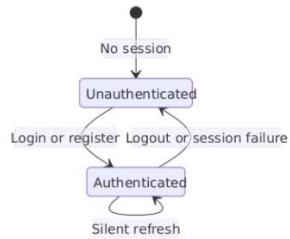
- **Clean Architecture** – Maintain strict layer separation with no reverse dependencies; Domain depends only on itself/BCL.
- **Explicit Mapping** – Use manual entity/DTO mapping; no AutoMapper.
- **Strict TypeScript** – Enable `strict`, `noImplicitAny`, and `strictNullChecks`.
- **Thin Controllers** – Controllers must not contain business or database/query logic.
- **Centralized Exception Handling** – Return a consistent JSON error envelope without exposing stack traces or internal details.
- **Security & Authentication** – Use JWT authentication with short-lived access tokens, HttpOnly refresh-token cookies, hashed refresh tokens, and token revocation.
- **Authorization** – Enforce RBAC and ownership-based authorization through JWT policies.
- **Rate Limiting** – Apply a fixed-window limit of 10 requests per minute to authentication endpoints.
- **CORS** – Restrict development cross-origin access to `http://localhost:4200` with credentials enabled.
- **Health Check** – Expose `/health` for application health monitoring.
- **DTO-Only API** – Do not expose domain or persistence entities directly through API responses.
- **Automatic Audit Fields** – Automatically populate `CreatedBy`, `CreatedAt`, `ModifiedBy`, and `ModifiedAt`.
- **Global Soft-Delete Filter** – Exclude logically deleted records from normal queries through a global query filter.
- **Redis Output Caching** – Use Redis-backed output caching for appropriate read-heavy endpoints.
- **Timezone** – Use `Asia/Riyadh` as the application's configured timezone.

ERD

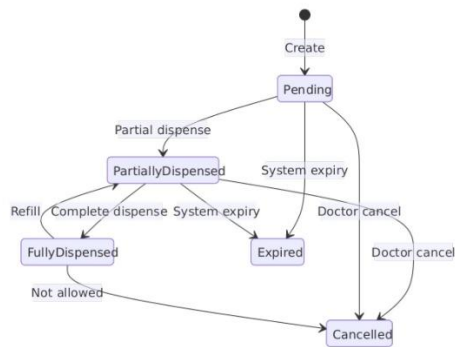


State Machine Diagram

1. Authentication & Session State Flow



2. Prescription Lifecycle State Flow



3. Medicine Batch Lifecycle State Flow

